

ArrayList vs LinkedList

vamos a devolver una explicación de la comparación entre ArrayList y LinkedList y luego devolver la explicación de las implementaciones LIFO y FIFO con Queue y DeQueue usando objetos LinkedList. Explicar que es posible hacer implementaciones FIFO y LIFO con List y ArrayList pero lo natural es hacerlo con objetos LinkedList que consuman Queue y Dequeue

Comparación entre ArrayList y LinkedList en Java

En Java, tanto ArrayList como LinkedList son implementaciones de la interfaz Java Collections Framework List, lo que significa que permiten almacenar colecciones ordenadas de elementos y acceder a ellos mediante índices.

Sin embargo, internamente funcionan de manera diferente y eso impacta directamente en el rendimiento y en los casos de uso recomendados.

ArrayList

ArrayList utiliza internamente un **array dinámico**.

Cuando agregamos elementos, Java reserva un bloque continuo de memoria. Si el array se llena, automáticamente crea uno más grande y copia los elementos.

Ventajas

- Acceso extremadamente rápido por índice (`get(i)`).
- Muy eficiente para recorridos.
- Menor consumo de memoria.
- Ideal cuando predominan las lecturas.

Desventajas

- Insertar o eliminar elementos en el medio puede ser costoso.
- Al agregar muchos elementos puede necesitar redimensionar el array.

Ejemplo

`</>` Java

```
List<String> nombres = new ArrayList<>();
```

```
nombres.add("Ana");  
nombres.add("Luis");  
nombres.add("Pedro");  
  
System.out.println(nombres.get(1)); // Luis
```

LinkedList

`LinkedList` utiliza internamente una **lista doblemente enlazada**.

Cada nodo almacena:

- el dato
- referencia al nodo siguiente
- referencia al nodo anterior

Por eso no necesita mover elementos al insertar o eliminar.

Ventajas

- Inserciones y eliminaciones rápidas.
- Muy útil para estructuras dinámicas.
- Excelente para colas y pilas.

Desventajas

- Acceso por índice más lento.
- Mayor consumo de memoria por los enlaces.

Ejemplo

```
<> Java  
  
List<String> nombres = new LinkedList<>();  
  
nombres.add("Ana");  
nombres.add("Luis");  
nombres.add("Pedro");
```

Diferencia conceptual importante

ArrayList

Piensa en:

- un vector
- un arreglo continuo en memoria

Acceder al elemento 500 es inmediato.

LinkedList

Piensa en:

- una cadena de nodos enlazados

Para llegar al elemento 500 hay que recorrer los nodos.

¿Cuándo conviene usar cada uno?

Situación	Estructura recomendada
Muchas lecturas por índice	ArrayList
Muchas inserciones/eliminaciones	LinkedList
Implementar colas	LinkedList
Implementar pilas	LinkedList
Uso general y acceso rápido	ArrayList

Implementaciones FIFO y LIFO

Las estructuras FIFO y LIFO representan distintas formas de administrar elementos.

FIFO — First In First Out

El primer elemento en entrar es el primero en salir.

Se comporta como:

- una fila
- una cola de atención

Ejemplo conceptual

ENTRA: A B C

SALE: A B C

LIFO — Last In First Out

El último elemento en entrar es el primero en salir.

Se comporta como:

- una pila de platos

Ejemplo conceptual

ENTRA: A B C

SALE: C B A

FIFO usando Queue y LinkedList

La interfaz `Queue` representa naturalmente una cola FIFO.

La clase `LinkedList` implementa la interfaz `Queue`.

Ejemplo

`</>` Java

```
import java.util.LinkedList;
import java.util.Queue;

public class FifoEjemplo {

    public static void main(String[] args) {

        Queue<String> cola = new LinkedList<>();

        cola.offer("Ana");
        cola.offer("Luis");
        cola.offer("Pedro");

        System.out.println(cola.poll()); // Ana
        System.out.println(cola.poll()); // Luis
        System.out.println(cola.poll()); // Pedro
    }
}
```

```
}  
}
```

Métodos típicos de Queue

Método	Función
<code>offer()</code>	agrega elemento
<code>poll()</code>	elimina y retorna el primero
<code>peek()</code>	consulta el primero
<code>isEmpty()</code>	verifica si está vacía

LIFO usando Deque y LinkedList

La interfaz `Deque` (Double Ended Queue) permite insertar y eliminar elementos por ambos extremos.

Gracias a esto puede utilizarse como una pila LIFO.

`LinkedList` también implementa `Deque`.

Ejemplo

Java

```
import java.util.Deque;  
import java.util.LinkedList;  
  
public class LifoEjemplo {  
  
    public static void main(String[] args) {  
  
        Deque<String> pila = new LinkedList<>();  
  
        pila.push("Ana");  
        pila.push("Luis");  
        pila.push("Pedro");  
  
        System.out.println(pila.pop()); // Pedro  
        System.out.println(pila.pop()); // Luis  
        System.out.println(pila.pop()); // Ana  
    }  
}
```

```
}  
}
```

Métodos típicos de Deque como pila

Método	Función
<code>push()</code>	apila elemento
<code>pop()</code>	desapila
<code>peek()</code>	consulta el tope

¿Por qué usar LinkedList para FIFO y LIFO?

Aunque técnicamente es posible implementar:

- colas FIFO
- pilas LIFO

utilizando `List` y `ArrayList`, no es la opción más natural ni eficiente.

Ejemplo FIFO con ArrayList

Java

```
List<String> cola = new ArrayList<>();  
  
cola.add("A");  
cola.add("B");  
cola.add("C");  
  
cola.remove(0);
```

El problema es que:

- al eliminar el primer elemento
- todos los demás deben desplazarse

Eso tiene un costo adicional.

Ejemplo LIFO con ArrayList

```
</> Java  
  
List<String> pila = new ArrayList<>();  
  
pila.add("A");  
pila.add("B");  
  
pila.remove(pila.size() - 1);
```

Esto sí funciona razonablemente bien, pero conceptualmente no expresa tan claramente la idea de pila.

Lo natural en Java

Para FIFO

```
</> Java  
  
Queue<String> cola = new LinkedList<>();
```

Porque:

- la interfaz `Queue` ya modela una cola
 - `LinkedList` está optimizada para inserciones y extracciones
-

Para LIFO

```
</> Java  
  
Deque<String> pila = new LinkedList<>();
```

Porque:

- `Deque` modela una estructura de doble extremo
 - puede comportarse naturalmente como pila
-

Idea importante de diseño

En Java normalmente se programa contra interfaces.

Por eso se recomienda:

```
</> Java  
  
Queue<String> cola = new LinkedList<>();  
Deque<String> pila = new LinkedList<>();
```

```
List<String> lista = new ArrayList<>();
```

y no:

</> Java

```
LinkedList<String> cola = new LinkedList<>();
```

De esta manera:

- el código queda más desacoplado
- más flexible
- más mantenible
- y permite cambiar implementaciones fácilmente sin modificar la lógica del programa.



me faltó incluir en la explicación que hemos trabajado hasta ahora con una aplicación que generaba una lista enlazada casera a través del concepto de nodo, ahora esto ya no es necesario porque Java provee directamente la clase ArrayList que internamente ya implementa los nodos

Relación con la implementación manual de listas enlazadas

Hasta ahora hemos trabajado construyendo una **lista enlazada casera**, utilizando explícitamente el concepto de nodo.

Por ejemplo, creábamos una clase `Nodo` que contenía:

- un dato
- una referencia al siguiente nodo

</> Java

```
public class Nodo {  
  
    private int dato;  
    private Nodo siguiente;  
  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.siguiente = null;  
    }  
  
    public int getDato() {  
        return dato;  
    }  
}
```



```
public Nodo getSiguiente() {  
    return siguiente;  
}  
  
public void setSiguiente(Nodo siguiente) {  
    this.siguiente = siguiente;  
}  
}
```

Luego construíamos manualmente la lista:

```
Nodo1 -> Nodo2 -> Nodo3 -> null
```

Esto fue muy importante desde el punto de vista pedagógico porque permitió comprender:

- cómo funciona internamente una lista enlazada
- el concepto de referencia
- cómo se enlazan objetos en memoria
- inserciones y eliminaciones
- recorridos nodo por nodo
- estructuras dinámicas

¿Qué cambia ahora?

A partir de las colecciones de Java ya no necesitamos implementar manualmente los nodos.

El Java Collections Framework ya provee estructuras listas para usar.

Por ejemplo:

```
<> Java  
LinkedList<String> lista = new LinkedList<>();
```

Internamente, `LinkedList` ya implementa:

- nodos
- enlaces
- recorridos
- inserciones
- eliminaciones

Todo eso ocurre automáticamente.

Aclaración importante sobre ArrayList

`ArrayList` NO implementa nodos internamente.

`ArrayList` utiliza un **array dinámico**.

La estructura que internamente trabaja con nodos es:

```
</> Java
LinkedList
```

Por lo tanto:

Clase	Implementación interna
<code>ArrayList</code>	array dinámico
<code>LinkedList</code>	nodos enlazados

Entonces... ¿para qué aprendimos nodos?

Porque comprender la implementación manual permite entender:

- cómo funcionan realmente las colecciones
- diferencias de rendimiento
- complejidad de operaciones
- por qué algunas estructuras son más eficientes para ciertos problemas

Por ejemplo:

- en una lista enlazada agregar elementos es simple porque solo se modifican referencias
- en un array puede requerirse mover elementos

Evolución natural del aprendizaje

Primera etapa

Implementación manual:

```
Nodo -> Nodo -> Nodo
```

Objetivo:

- comprender estructuras dinámicas

Segunda etapa

Uso de colecciones ya implementadas:

Java

```
List<String> lista = new LinkedList<>();
```

O

Java

```
List<String> lista = new ArrayList<>();
```

Objetivo:

- reutilizar estructuras optimizadas
- escribir código más limpio
- aprovechar bibliotecas estándar de Java

Idea importante

En el desarrollo profesional normalmente NO se implementan listas enlazadas desde cero.

Se utilizan las colecciones del framework de Java porque:

- están optimizadas
- fueron probadas exhaustivamente
- reducen errores
- simplifican el desarrollo

Sin embargo, entender cómo funcionan internamente sigue siendo fundamental para:

- algoritmos
- estructuras de datos
- análisis de eficiencia
- entrevistas técnicas
- diseño de software.

